Java Immutable Classes — In-Depth Notes

---

1. What is an Immutable Object?

An immutable object is one whose **state cannot be changed after it is created**. Once you create the object and set its values, those values are locked forever — no one can modify them.

Every object has two things:

- **Instance variables** (data/state)
- **Methods** (behaviour)

An immutable object guarantees that neither its variables nor its behaviour can be changed after creation.

A simple analogy — imagine you wrote something in permanent marker. You can read it, you can copy it, but you can never erase or overwrite it.

---

2. Why Do We Need Immutability?

The biggest use case is **multithreading**. When multiple threads run simultaneously and share the same object, they can accidentally overwrite each other's changes. This is called a **race condition**.

Immutable objects completely eliminate this problem because no thread can ever modify the object. Every thread just reads it safely.

Other benefits:

- Simpler, more predictable code
- Safer to share objects across different parts of your program
- No need for defensive locking in concurrent code

---

3. A Normal (Mutable) Class — The Problem

Let's say you have this typical Student class:

```
class Student {
```

```
    int age;
    String name;

    Student(int age, String name) {
        this.age = age;
        this.name = name;
    }

    // getters
    public int getAge() { return this.age; }
    public String getName() { return this.name; }

    // setters
    public void setAge(int age) { this.age = age; }
    public void setName(String name) { this.name = name; }
}
```

Now if you do:

```
Student s1 = new Student(28, "Aditya");
s1.setName("Rohit");   // ✅ works — name changed!
s1.setAge(30);         // ✅ works — age changed!
```

This is completely mutable — the object can be changed freely after creation. Not immutable at all.

Also, someone could create a subclass and override its methods:

java

```
class CSCStudent extends Student {
    @Override
    void markAttendance() {
        // completely different behaviour!
    }
}
```

This changes the **behaviour** of the original class too. So mutability applies to both state and behaviour.

---

4. Rules to Make a Class Immutable

From the example above, you can figure out exactly what needs to be locked down:

Rule 1 — Mark the class as `final`

This prevents anyone from extending (inheriting) the class. If no one can subclass it, no one can override its methods and change its behaviour.

```
final class Student { ... }
```

If you try to do `class CSCStudent extends Student`, you'll get a compile error: *"The type CSCStudent cannot subclass the final class Student"*.

Rule 2 — Mark all instance variables as `private` and `final`

- `private` — no one can directly access them from outside the class
- `final` — once assigned a value, that value can never be changed

```
private final int age;
private final String name;
```

Rule 3 — No setters

Remove all setter methods completely. If there's no `setName()` or `setAge()`, there's no way to change the values from outside.

Rule 4 — Initialize everything through the constructor only

Since variables are `final`, they must be assigned exactly once. The constructor is the only place to do this.

```
Student(int age, String name) {
   this.age = age;
   this.name = name;
}
```

After the constructor runs, those values are locked forever.

Rule 5 (Advanced) — Defensive copy for non-primitive fields

This rule is the tricky one — explained in full detail below.

---

5. Basic Immutable Class in Code

Applying the first four rules:

```
final class Student {

    private final int age;
    private final String name;

    Student(int age, String name) {
        this.age = age;
        this.name = name;
    }

    public int getAge() { return this.age; }
    public String getName() { return this.name; }

    // NO setters!
}
```

Now:

```
Student s1 = new Student(28, "Aditya");

// s1.setName("Rohit");  ❌ — no such method
// s1.name = "Rohit";    ❌ — private, can't access directly

System.out.println(s1.getName()); // ✅ Aditya
System.out.println(s1.getAge());  // ✅ 28
```

This works perfectly. The object is immutable as long as all fields are **primitive types** (`int`, `double`, `boolean`, etc.) or **Strings** (which are immutable by default in Java).

---

6. The Problem with Non-Primitive Fields (Object References)

Now here's where it gets interesting — and this is a very common interview question.

What if one of the fields is not a primitive, but an **object**? For example, instead of storing the college name as a `String`, you have a whole `College` class:

```
class College {
    String name;
    String address;
```

```
    College(String name, String address) {
        this.name = name;
        this.address = address;
    }
}
```

And your Student class now looks like this:

```
final class Student {

    private final int age;
    private final String name;
    private final College college;  // <-- object reference!

    Student(int age, String name, College college) {
        this.age = age;
        this.name = name;
        this.college = college;
    }

    public int getAge() { return this.age; }
    public String getName() { return this.name; }
    public College getCollege() { return this.college; }
}
```

Looks correct right? Final, private, no setters. But watch what happens:

```
College c = new College("IIT Guwahati", "Assam");
Student s1 = new Student(28, "Aditya", c);

// Let's print the college name
System.out.println(s1.getCollege().name); // prints: IIT Guwahati

// Now let's "attack" the object
s1.getCollege().name = "IIT Bombay";

// Print again
System.out.println(s1.getCollege().name); // prints: IIT Bombay  !!!
```

The college name changed! Even though the class had `private final` on the college field, and no setters! The class is **NOT truly immutable**.

---

7. Why Did That Happen? — Understanding Shallow Copy

To understand this, you need to think about how Java stores objects in memory.

Java memory has two parts:

- **Stack memory** — stores reference variables (pointers)
- **Heap memory** — stores actual objects

When you wrote `College c = new College("IIT Guwahati", "Assam")`:

- A College object was created in the **heap**
- A reference variable `c` was created in the **stack**, pointing to that heap object

When you wrote `new Student(28, "Aditya", c)` and inside the constructor did `this.college = college`:

- You just copied the **reference** `c` into `this.college`
- Both `c` and `this.college` now point to the **same heap object**

So now you have **two pointers pointing to one object**.

When you call `s1.getCollege()`, the getter returns `this.college` — which is that same reference pointer. Now the caller has a direct pointer to your internal College object. They can go into it and change `name`, `address`, anything they want. And since it's the same object in heap memory, your Student's college also reflects the change.

This is called a **shallow copy** — you copied the reference/pointer instead of creating a new independent object.

---

8. The Solution — Defensive Copy (Deep Copy)

The fix is to **never expose your internal object's reference**. Whenever you receive or return a non-primitive object, create a brand new copy of it.

This is called a **defensive copy**.

You need to do this in two places:

Place 1 — In the constructor (when receiving the object)

Instead of blindly storing the reference that was passed in, create a new object:

```
Student(int age, String name, College college) {
    this.age = age;
    this.name = name;
    // Don't do: this.college = college;
    // Instead, create a NEW College object:
    this.college = new College(college.name, college.address);
}
```

Now even if someone passes you a College reference and later modifies that original College object, your internal copy is completely separate and unaffected.

Place 2 — In the getter (when returning the object)

Instead of returning the actual internal reference, return a new copy:

```
public College getCollege() {
    // Don't do: return this.college;
    // Instead, return a NEW College object:
    return new College(this.college.name, this.college.address);
}
```

Now even if the caller modifies what `getCollege()` returns, they're modifying a fresh copy. Your original internal object stays untouched.

---

9. The Full Truly Immutable Student Class

```
final class Student {

    private final int age;
    private final String name;
    private final College college;

    Student(int age, String name, College college) {
        this.age = age;
        this.name = name;
        this.college = new College(college.name, college.address); // defensive copy
    }

    public int getAge() { return this.age; }

    public String getName() { return this.name; }
```

```
    public College getCollege() {
        return new College(this.college.name, this.college.address); // defensive copy
    }
}
```

Now let's test it:

```
College c = new College("IIT Guwahati", "Assam");
Student s1 = new Student(28, "Aditya", c);

System.out.println(s1.getCollege().name); // IIT Guwahati

// Try to attack it
s1.getCollege().name = "IIT Bombay";

// Print again
System.out.println(s1.getCollege().name); // Still: IIT Guwahati ✅
```

The original object was never touched. The attack only modified the temporary copy that `getCollege()` handed out.

---

10. What Happened Behind the Scenes (Memory Diagram in Words)

When `new Student(28, "Aditya", c)` was called:

1. A new Student object was created in heap
2. Inside the constructor, `new College(college.name, college.address)` ran — this created a **second, separate** College object in heap with the same values (IIT Guwahati, Assam)
3. `this.college` inside Student points to this **second** object, NOT to the original `c`

So now in heap you have:

- College object 1 — pointed to by `c` — "IIT Guwahati", "Assam"
- College object 2 — pointed to by `s1`'s internal college field — "IIT Guwahati", "Assam"
- Student object — pointed to by `s1`

When someone calls `s1.getCollege()`:

- A **third** College object is created in heap — "IIT Guwahati", "Assam"
- That third object is handed to the caller

- The caller can do whatever they want with it — it has zero connection to the Student's internal College object

The Student's internal College object is **never shared with anyone, ever**. That's true immutability.

---

11. Shallow Copy vs Deep Copy — Summary

	Shallow Copy	Deep Copy
What gets copied	Only the reference/pointer	A brand new object is created
Both variables point to	Same object in heap	Different objects in heap
Change in one affects other?	Yes	No
Used for immutability?	No — breaks it	Yes — required for it

Shallow copy is like giving someone your house key — they can walk in and rearrange your furniture.

Deep copy is like building them an identical copy of your house — they can do whatever they want to theirs, your house stays exactly as it was.

---

12. What About Strings?

You may have noticed that `String name` is also non-primitive (it's an object), but we didn't apply defensive copy to it. Why?

Because **Strings in Java are immutable by default**. It's baked into the language. You can never modify a String object once it's created. When you do `str = str + "abc"`, Java isn't modifying the original String — it's creating a brand new String object and reassigning the reference.

So Strings are already safe. You only need defensive copies for your own mutable classes like `College`.

---

13. Two Approaches to Handle Mutable Non-Primitive Fields

Approach 1 — Make the referenced class also immutable

If you make `College` immutable too (final class, private final fields, no setters), then even if you return `this.college` directly, the caller can't change anything inside it. Problem solved at the source.

Approach 2 — Defensive copy (as explained above)

If you can't touch the `College` class (maybe it's from a third-party library), then do defensive copies in your constructor and getter. This is the more general, universally applicable approach.

---

14. Complete Rules Checklist for Immutable Class

- `final` keyword on the class → prevents inheritance and method overriding
- `private` on all instance variables → prevents direct external access
- `final` on all instance variables → prevents reassignment after construction
- No setters → no way to change values after object creation
- Initialize all fields through constructor only
- For any non-primitive / object type fields → apply **defensive copy** in both the constructor (when receiving) and the getter (when returning)
- Primitive fields (`int`, `double`, `boolean`, etc.) → automatically safe, no extra work needed
- `String` fields → already immutable in Java, no defensive copy needed

---

15. Real World Examples of Immutable Classes in Java

Java's standard library is full of immutable classes:

- `String` — most famous example
- `Integer`, `Long`, `Double` and all other wrapper classes
- `LocalDate`, `LocalTime` in the `java.time` package
- `BigDecimal`, `BigInteger`

These are all designed immutable by the Java creators for safety and thread-friendliness.

---

16. Quick Interview Revision

Q: What is an immutable class? A class whose objects, once created, cannot be changed in any way — neither their data nor their behaviour.

Q: How do you create one? Make class final, all fields private and final, no setters, constructor-only initialization, and defensive copy for object-type fields.

Q: What is shallow copy? Copying only the reference pointer — both variables point to the same heap object.

Q: What is deep copy? Creating a brand new independent object — each variable points to its own separate heap object.

Q: Why are immutable objects important in multithreading? Because multiple threads can read them safely without any synchronization. Since no thread can modify the object, there's no risk of race conditions or data corruption.

Q: Why isn't `private final College college` enough for immutability? Because `final` only prevents reassigning the reference — it doesn't prevent modifying the object that the reference points to. You still need defensive copy.